

Relacionamentos entre Objetos & POO no Back-End

Organizando dados, regras e ciclos de vida no servidor

Uma análise técnica sobre como modelar conexões do mundo real em arquiteturas de software escaláveis.

GRUPO 4

Curso Técnico em Desenvolvimento de Sistemas

DISCIPLINA

Programação Back-End

DATA

Agosto 2026

Como transformar o mundo real em código confiável?



Relacionamentos

Associação, Agregação e Composição como ferramentas de modelagem.



Ciclo de Vida

Entender o que sobrevive quando o "todo" desaparece.



Arquitetura Back-End

Separar Entidades (dados) de Serviços (regras de negócio).



Aplicação Prática

Implementar um fluxo de e-commerce em JavaScript moderno.

Mensagem-chave: Modelar bem evita dados inconsistentes no futuro.

Associação, Agregação e Composição



ASSOCIAÇÃO

Ligação simples e livre. Os objetos são independentes e apenas se conhecem mutuamente.

EXEMPLO

Cliente ↔ Pedido



AGREGAÇÃO

Relação "todo-parte" fraca. A parte pode existir separadamente fora do objeto principal.

EXEMPLO

Time → Jogadores



COMPOSIÇÃO

Relação "todo-parte" forte. A parte pertence ao todo e depende dele para existir.

EXEMPLO

Pedido → Itens do Pedido

A Pergunta que Separa Agregação de Composição

"Se o objeto principal for eliminado, a parte continua a fazer sentido e a existir de forma independente?"

RELAÇÃO	DEPENDÊNCIA	SE O "TODO" DESAPARECER...
Associação	Baixa	O outro objeto continua a existir de forma totalmente independente.
Agregação	Parcial	As partes sobrevivem e podem ser reutilizadas ou associadas a outro grupo.
Composição	Total	As partes são eliminadas em cascata juntamente com o objeto principal.

POO no Back-End: Entidades e Serviços

Entidades do Domínio

Representam os conceitos reais do negócio e guardam o estado e os dados do sistema.

- Cliente
- Pedido
- ItemPedido
- Produto

Classes de Serviço

Coordenam as ações, concentram a lógica e aplicam as regras de negócio complexas.

- PedidoService
- UsuarioService
- PagamentoService

ENTRADA
Requisição



LÓGICA
Service



DADOS
Entidades



SAÍDA
Persistência

Do Conceito para o Banco de Dados

As relações na memória tornam-se regras de persistência

Associação

Gera **Chaves Estrangeiras (FK)**. O `clienteId` liga o pedido ao cliente sem misturar os seus ciclos de vida.

Agregação

Entidades independentes. Jogadores podem ficar "sem time" ou ser transferidos sem serem eliminados.

Composição

Define **Exclusão em Cascata**. Se o Pedido for removido, os itens vinculados são eliminados automaticamente.

TABLE: Pedido

id (PK)

cliente_id (FK)

data_criacao

↓ CASCADE DELETE

TABLE: ItemPedido

id (PK)

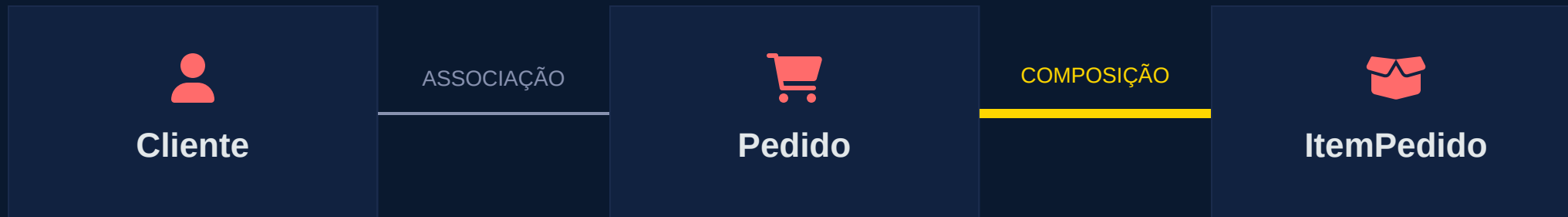
pedido_id (FK)

produto_id

quantidade

 Regra de Ouro: Modelar corretamente evita a criação de **dados órfãos** no servidor.

O Nosso Cenário de E-commerce



PedidoService

Valida Stock • Constrói Entidades • Persiste no Banco de Dados

■ Relação Independente

■ Ciclo de Vida Dependente

Implementação das Entidades

```
class ItemPedido {
  constructor(nome, preco) {
    this.nome = nome;
    this.preco = preco;
  }
}

class Pedido {
  constructor(id, clienteId) {
    this.id = id;
    this.clienteId = clienteId; // Associação
    this.itens = [];           // Composição
  }

  adicionarItem(nome, preco) {
    const item = new ItemPedido(nome, preco);
    this.itens.push(item);
  }
}
```

ASSOCIAÇÃO

O `clienteId` liga o pedido ao cliente apenas por referência. O ciclo de vida do Cliente é independente do Pedido.

COMPOSIÇÃO

O uso de `new ItemPedido` dentro da classe Pedido torna o Pedido "dono" do ciclo de vida dos itens.

* Exemplo simplificado para fins didáticos.

O Serviço Aplica as Regras

```
class PedidoService {
  constructor() {
    this.bancoDeDadosSimulado = [];
  }

  processarNovoPedido(id, clienteId, itensCarrinho) {
    const pedido = new Pedido(id, clienteId);

    for (const item of itensCarrinho) {
      if (item.preco <= 0) {
        throw new Error("Preço inválido");
      }
      pedido.adicionarItem(item.nome, item.preco);
    }

    this.bancoDeDadosSimulado.push(pedido);
    return pedido;
  }
}
```

COORDENAÇÃO

O serviço não guarda dados permanentemente; ele orquestra a criação das entidades e a sua validação.

REGRAS DE NEGÓCIO

Centraliza validações (ex: preço > 0) antes de permitir que os dados cheguem à camada de persistência.

 O serviço garante que o sistema nunca entre num estado inválido.

O que acontece em cada etapa?

1

RECEBER

O serviço recebe os dados brutos (JSON) da requisição: IDs e itens.

2

VALIDAR

Aplica regras de negócio, como impedir preços negativos ou stock insuficiente.

3

CONSTRUIR

Cria a entidade Pedido e instancia os ItemPedido internamente (Composição).

4

PERSISTIR

Guarda o objeto completo no banco de dados após todas as validações.

Nota Técnica: A palavra-chave **this** aponta para o objeto em execução, garantindo isolamento. O operador **new** dentro do serviço torna explícita a criação e o controle do ciclo de vida das entidades.

Conclusão e Fontes

O QUE DEVEMOS LEMBRAR?

- ✓ **Modelagem de Relações**
Associação (independente), Agregação (fraca) e Composição (forte) ditam a integridade dos dados.
- ✓ **Ciclo de Vida**
Entender a dependência entre objetos evita a criação de dados órfãos no banco de dados.
- ✓ **Arquitetura Back-End**
A separação entre Entidades e Serviços garante um código escalável e regras de negócio centralizadas.

Fontes Consultadas

MDN Web Docs: Classes

JavaScript Reference

MDN Web Docs: Working with Objects

JavaScript Guide

MDN Web Docs: Private Elements

JavaScript Reference

Materiais Didáticos

PDFs 01 e 02 de Apoio ao Aluno

Alguma dúvida?